

Insights clave de Andrej Karpathy sobre agentes LLM y orquestacion

Documento ejecutivo para disenar sistemas agentic confiables

Preparado para: Lalo Sanchez | Fecha: 5 de mayo de 2026

Tesis central

Karpathy no esta vendiendo la fantasia de agentes completamente autonomos. Su lectura es mas sobria: estamos entrando a Software 3.0, donde los LLMs son programables con lenguaje natural, pero los productos serios requieren contexto curado, herramientas, memoria explicita, supervision humana y niveles graduales de autonomia.

Este documento sintetiza ideas de la charla Software Is Changing (Again), posts de Karpathy, discusiones sobre context engineering, y entrevistas/reportes recientes. No todas las frases son citas literales; la mayor parte es una interpretacion operacional para arquitectura de agentes.

Contenido

- Resumen ejecutivo
- Insights principales
- Arquitectura recomendada
- Checklist de implementacion
- Anti-patterns
- Aplicacion a un producto tipo Quasar
- Fuentes consultadas

1. Resumen ejecutivo

Idea	Lectura practica
Software 3.0	Los LLMs son una nueva capa programable mediante lenguaje natural. El prompt se parece mas a codigo de control que a copywriting.
Context engineering	La ventaja real no esta en prompts bonitos, sino en decidir que contexto, estado, herramientas, ejemplos y restricciones entran en cada paso.
Autonomia parcial	La unidad correcta de diseno no es agente 100% autonomo; es un slider de autonomia: sugerir, editar, ejecutar con aprobacion, ejecutar de forma vigilada.
Humano en el loop	La AI genera; el humano verifica. El producto debe hacer esa verificacion rapida, visual y segura.
Agentes como interns	Los agentes son utiles, pero aun tienen inteligencia irregular, poca memoria continua, problemas de computer use y errores compuestos.
Build for agents	El software empieza a necesitar interfaces legibles para LLMs: Markdown, APIs claras, schemas, logs, comandos reproducibles y protocolos como MCP.

2. Insights principales

2.1 Software 3.0: programar en lenguaje natural

Karpathy describe una tercera etapa del software: primero escribiamos codigo explicito, luego entrenabamos redes neuronales como software 2.0, y ahora usamos LLMs programables con prompts en lenguaje natural. Para agentes, esto significa que instrucciones, restricciones, ejemplos y contexto son parte del programa, no decoracion. [1][2]

2.2 El LLM como sistema operativo

Karpathy usa la analogia de los LLMs como una nueva computadora: el modelo se parece a una CPU y la ventana de contexto a una RAM limitada. La app agentic seria el sistema que administra memoria, herramientas, estado y ejecucion. [1][2]

2.3 Context engineering supera a prompt engineering

Karpathy apoya explicitamente el termino context engineering: en apps industriales, el trabajo dificil es llenar la ventana de contexto con la informacion correcta para el siguiente paso: tarea, ejemplos, RAG, datos multimodales, herramientas, estado, historial y compactacion. [3][4]

2.4 Inteligencia irregular: mucho poder, fallas raras

Los LLMs pueden producir resultados impresionantes y a la vez fallar en cosas basicas. Por eso no se deben disenar como empleados senior autonomos, sino como colaboradores rapidos que requieren restricciones, evidencia y supervision.

2.5 Autonomy slider: disenar niveles de autonomia

Karpathy enfatiza apps con autonomia parcial: Cursor tiene niveles que van de autocompletado a modo agente; Perplexity escala de busqueda a deep research; Tesla Autopilot tambien comunica niveles de autonomia. El producto debe permitir mover ese slider segun riesgo y confianza. [5]

2.6 Generation-verification loop

La AI genera y el humano verifica. El loop ganador reduce el costo de verificacion: diffs, previews, fuentes, logs, botones de aceptar/rechazar, rollback y tests. [5]

2.7 Keep the AI on a tight leash

La mejor practica no es pedir trabajos enormes; es dividir tareas, dar limites claros y pedir cambios pequenos y verificables. Esto baja el riesgo de diffs gigantes, errores ocultos y supuestos inventados. [5]

2.8 Demo-product gap

Karpathy remarca que una demo que funciona una vez no es un producto que funciona siempre. El estandar de producto requiere confiabilidad, no solo una secuencia bonita de tool calls. [5]

2.9 Decada de agentes, no ano de agentes

Karpathy ha sido critico con la idea de que 2025 fuese simplemente 'el ano de los agentes'. Su tesis es que faltan capacidades profundas: inteligencia, multimodalidad, uso confiable de computadoras y aprendizaje continuo. [6][7]

2.10 Taste, judgement y supervision siguen siendo humanos

Incluso en coding agents y vibe coding, Karpathy ha dicho que los humanos siguen a cargo de estetica, juicio, gusto y oversight. Tambien ha criticado que mucho codigo generado por AI aun sea bloaty, repetitivo y brittle. [8]

2.11 Build for agents

Karpathy propone pensar en agentes como nuevos consumidores de informacion digital. Esto empuja hacia docs en Markdown, APIs claras, comandos reproducibles, schemas y protocolos de conexion de herramientas/datos como MCP. [1][9]

La frase util para recordar

No construyas un 'agente magico'. Construye un sistema operativo pequeno alrededor del LLM: contexto, herramientas, memoria, verificacion, permisos y niveles de autonomia.

3. Arquitectura recomendada para agentes LLM

Una arquitectura alineada con estos insights se parece mas a un pipeline controlado que a un chat libre:

- User goal
 - > Context engineering layer
 - tarea, restricciones, archivos, estado, memoria, ejemplos, RAG curado
 - > Planner / task decomposition
 - > Tool execution
 - APIs, browser, file system, code runner, MCP/tools
 - > Intermediate artifact
 - plan, diff, checklist, reporte, JSON, test plan
 - > Human verification UI
 - aceptar, rechazar, editar, inspeccionar fuentes, rollback
 - > Controlled execution
 - autonomia parcial con permisos y limites

4. Componentes que debería tener un buen sistema agentic

Componente	Responsabilidad
Context manager	Selecciona contexto minimo viable: archivos, historial, estado, fuentes, ejemplos, restricciones y datos relevantes.
Tool router	Decide que herramienta usar y cuando: busqueda, repo, API, navegador, calendario, base de datos, etc.
Memory layer	Distingue memoria de sesion, memoria persistente y estado derivado. No supone que el modelo recordara todo.
Verification layer	Obliga al agente a mostrar evidencia, fuentes, diffs, logs y criterios de aceptacion.
Permission layer	Controla acciones destructivas o sensibles: escribir archivos, mandar emails, correr comandos, hacer deploys.
Autonomy slider	Permite elegir entre sugerir, modificar, ejecutar con aprobacion o ejecutar de forma monitorizada.
Rollback / recovery	Permite revertir cambios, reintentar con contexto mejorado y auditar decisiones.

5. Slider de autonomia: una escala practica

- **Nivel 0 - Solo explica:** El modelo responde, pero no toca nada. Bueno para exploracion, aprendizaje y debugging conceptual.
- **Nivel 1 - Sugiere cambios:** Devuelve plan, criterios y propuesta. El humano decide si aplica.
- **Nivel 2 - Prepara artefacto:** Genera diff, PR, checklist, reporte o spec. No ejecuta acciones finales.
- **Nivel 3 - Ejecuta con aprobacion:** Puede usar herramientas, pero pide confirmacion antes de acciones sensibles.
- **Nivel 4 - Ejecuta monitorizado:** Corre flujos de bajo riesgo con logs, limites, timeouts y rollback.
- **Nivel 5 - Autonomia amplia:** Solo para entornos muy acotados, con pruebas fuertes y consecuencias controladas.

6. Checklist de implementacion

- Cada tarea del agente tiene un objetivo explicito y criterios de exito.
- El contexto se arma por tarea; no se inyecta todo el repositorio o toda la base documental sin curacion.
- El agente sabe que herramientas puede usar y cuales estan prohibidas.
- Las acciones de alto riesgo requieren aprobacion humana.
- Toda respuesta importante muestra evidencia: fuentes, logs, comandos, diffs o razonamiento verificable.

- El usuario puede aceptar/rechazar cambios por bloque, no solo todo o nada.
- Existe rollback o forma clara de recuperar estado previo.
- Los errores se muestran como estados de producto, no como texto opaco.
- La UI permite inspeccionar el contexto usado y las herramientas llamadas.
- Hay tests o validaciones automáticas antes de considerar completada la tarea.
- La memoria persistente esta separada de la memoria temporal de sesion.
- Las instrucciones externas no confiables no pueden sobrescribir instrucciones internas.

7. Anti-patterns que debes evitar

- **Multi-agent por moda:** Agregar mas agentes aumenta coordinacion, latencia y debugging. Primero prueba un agente bien acotado.
- **RAG como basurero:** Meter demasiados documentos puede empeorar la respuesta. La clave es relevancia, no volumen.
- **Autonomia total prematura:** El agente que puede actuar sin limites tambien puede equivocarse sin limites.
- **Prompts gigantes sin estructura:** Un prompt largo no reemplaza estado, tools, memoria y validacion.
- **Sin evidencia:** Si el usuario no puede verificar lo que hizo el agente, no es un producto serio.
- **Sin rollback:** Si no puedes revertir, no deberias permitir que el agente escriba cambios importantes.
- **UI tipo chat para todo:** Muchas tareas agentic requieren diffs, tablas, previews, logs y botones, no solo burbujas de texto.

8. Aplicacion a un producto tipo Quasar / chatbot empresarial

Para un producto de chat empresarial con React, configuraciones NLP, fuentes, feedback, analisis y vistas simplificadas, yo lo aterrizaria asi:

- **Context packs por workflow:** Por ejemplo: feedback-persistence-pack, document-quality-pack, dictation-pack, analysis-pack. Cada pack incluye archivos relevantes, endpoints, tipos TS, restricciones y criterios de aceptacion.
- **UI de auditoria:** Mostrar que documentos/fuentes se usaron, que tool calls ocurrieron, que parametros fueron enviados y que supuestos hizo el agente.
- **Modo propuesta antes de ejecucion:** Para cambios de codigo o configuracion: primero plan + diff + riesgos; despues aplicar con aprobacion.
- **Agentes por tarea, no por personalidad:** No necesitas 'agente creativo', 'agente serio', etc. Necesitas workflows: generador de pruebas, analizador de fuentes, revisor de contratos API, generador de UI desde wireframes.
- **Verificacion visual:** Para UI, el output deberia incluir screenshot, estados vacios/loading/error, y checklist de accesibilidad basica.
- **Memoria y preferencias explicitas:** Lo que el usuario guarda debe vivir en una capa de memoria clara; no depender de que el modelo 'se acuerde'.

9. Patrones de instrucciones utiles

Ejemplo de instruccion para mantener al agente en una tarea pequena y verificable:

Objetivo: modificar solo MessageList.tsx para persistir visualmente el feedback despues del PATCH.

Restricciones:

- No cambiar contrato de API.
- No tocar estilos globales.
- No modificar componentes no relacionados.

Salida requerida:

1. Plan breve.
2. Diff propuesto.
3. Riesgos.
4. Checklist de verificacion manual.
5. Tests sugeridos.

10. Sintesis final

Decision de diseno

El moat no sera tener 'un agente'. El moat sera disenar mejor que otros su contexto, herramientas, permisos, verificacion, memoria y experiencia de supervision humana.

La lectura mas fuerte de Karpathy es que los agentes no reemplazan el diseno de software: lo vuelven mas importante. El desarrollador/product designer pasa de escribir cada linea a orquestar sistemas: decidir intencion, contexto, herramientas, validacion y nivel de autonomia. Eso requiere criterio tecnico, no solo entusiasmo por frameworks agentic.

Fuentes consultadas

- [1] **Y Combinator - Andrej Karpathy: Software Is Changing (Again)**. Pagina oficial de la charla en AI Startup School. <https://www.ycombinator.com/library/MW-andrej-karpathy-software-is-changing-again>
- [2] **The Singju Post - Transcript: Software Is Changing (Again)**. Transcripcion de la keynote del 18 de junio de 2025. <https://singjupost.com/andrej-karpathy-software-is-changing-again/>
- [3] **Karpathy en X - Context engineering over prompt engineering**. Post de Karpathy sobre context engineering. <https://x.com/karpathy/status/1937902205765607626>
- [4] **Simon Willison - Context engineering**. Cita y contextualizacion del post de Karpathy. <https://simonwillison.net/2025/jun/27/context-engineering/>
- [5] **Latent Space - Andrej Karpathy on Software 3.0**. Resumen estructurado de la charla, incluyendo autonomy sliders y generation-verification loop. <https://www.latent.space/p/s3>
- [6] **Simon Willison - Andrej Karpathy: AGI is still a decade away**. Notas sobre la conversacion con Dwarkesh y la tesis de la decada de agentes. <https://simonwillison.net/2025/Oct/18/agi-is-still-a-decade-away/>
- [7] **Business Insider - It will take a decade before AI agents actually work**. Reporte sobre las criticas de Karpathy al estado actual de los agentes. <https://www.businessinsider.com/andrej-karpathy-ai-agents-timelines-openai-2025-10>
- [8] **Business Insider - AI code can still be awkward and gross**. Reporte sobre sus comentarios recientes de coding agents, oversight y calidad de codigo. <https://www.businessinsider.com/andrej-karpathy-vibe-coding-ai-code-awkward-gross-needs-humans-2026-4>
- [9] **Anthropic - Introducing the Model Context Protocol**. Referencia sobre MCP como infraestructura para conectar modelos con datos y herramientas. <https://www.anthropic.com/news/model-context-protocol>